



Your AI assistant that runs entirely on your own machine.

PRIVATE BY DESIGN · OFFLINE-FIRST · DESKTOP-NATIVE

Raghav Gangwar · raghavgangwar222@gmail.com · Source: github.com/SheinRG/Luna · Deliverable: `Luna.exe`
(Windows 10/11)

Luna is a local-first AI desktop assistant for Windows. It chats, remembers what matters, and actually *does* things on your computer — opening any installed app, finding files, controlling the PC, tidying folders, setting reminders — with a permission prompt before every action. **No cloud. No accounts. Nothing leaves your laptop.** The language model runs locally through Ollama; the entire app works with the network cable unplugged.

What Luna does

- **Chat** — streamed, markdown-rendered replies, file & image upload, history, stop-generation.
- **Personal memory** — learns preferences via semantic recall (embeddings); every memory is reviewable, editable, deletable.
- **Open anything** — resolves classic apps, Microsoft Store/UWP apps (via AppUserModelID), and ~50 Windows Settings pages (`ms-settings:` deep links), with a bounded disk hunt as fallback.
- **PC control** — lock, screenshot, volume, brightness; shutdown / restart / empty Recycle Bin always require explicit confirmation.
- **Files** — natural-language file search across user folders; PDF/text summarization (map-reduce, fits a 4K context).
- **Productivity** — notes, to-dos, reminders (native Windows toasts), AI-drafted emails.
- **Organize Downloads** — previewed move plan, explicit confirmation, journaled moves with one-command *undo*.
- **Voice** — on-device Whisper speech-to-text; offline OS voices for read-aloud.
- **Privacy dashboard** — granted permissions, full action timeline, what's stored & where, delete-all.
- **Personalization** — assistant name, themes, 6 accent colors, 4 personalities, response length, memory on/off.

How the AI pipeline works

message → memory recall → intent router → (action | chat) → result

The router is two-layered: **deterministic regex fast-paths** catch common commands instantly with zero LLM cost, and an **LLM JSON classifier at temperature 0** handles everything else — same input, same route, every time. Low-confidence classifications fall back to plain chat rather than firing a wrong action.

Every action flows through **propose** → **user approval** → **execute** → **result**. Each action is declared in a registry with a permission category, read-only flag, and destructive flag. First use of a category asks permission; destructive operations *always* show a preview and require confirmation, regardless of stored permissions. System commands are a **fixed whitelist** — the model can only select one, never compose a shell string.

Architecture

```
Luna.exe (PyInstaller onedir)
└─ luna/main.py
   │ └─ FastAPI + uvicorn → 127.0.0.1:<free port> (background thread)
   │ └─ pywebview window → the UI (WebView2) (main thread)
   │   │ fallback: Edge --app=<url> → default browser
   │ └─ reminder scheduler → winotify toasts (background thread)
   │
   │ core/      ollama client · chat engine (SSE) · intent router · memory · voice
   │ actions/   apps · files · documents · productivity · system (permission-gated)
   │ server/    REST + SSE routes
   │ ui/        vanilla HTML/CSS/JS SPA, self-hosted fonts, fully offline
   │ db.py      SQLite (%LOCALAPPDATA%\Luna\luna.db)
```

Backend: Python 3.12, FastAPI, httpx, SQLite. **Frontend:** dependency-free HTML/CSS/JS single-page app; fonts vendored locally — no CDN. **Streaming:** Server-Sent Events; the chat stream stays open across the action-approval handshake.

Engineered for 8 GB RAM, CPU-only

The target machine is a modest laptop — no discrete GPU, every token generated on CPU. That constraint drove every model decision:

Choice	Reason
llama3.2:3b (chat, routing, summarizing)	The sweet spot for 8 GB CPU-only: ~2 GB resident, coherent, streams fast. 7B+ models swap and thrash.
nomic-embed-text (memory)	274 MB embedding model for semantic recall — coexists with the chat model without a second heavyweight load.
faster-whisper tiny.en (voice)	~75 MB, int8, lazy-loaded on first mic use and unloaded after 5 min idle — never competes with the LLM for RAM.
Ollama (runtime)	Local server, streaming API, structured JSON output, trivial model management.

Luna never loads two language models at once, caps context at 4096 tokens, and serializes background model calls.

Safety & privacy, plainly

- Zero outbound network requests during normal use — model, database, files, and fonts are all local.
- Permission asked per action category on first use; destructive actions always preview + confirm; Downloads organization is journaled and undoable.
- The Privacy dashboard shows every permission, every action taken, and everything stored — with one-button delete-all.
- All data lives in one place the user controls: `%LOCALAPPDATA%\Luna`.

Quality

- **72-check automated smoke suite** (`scripts/smoke.py`): intent routing, app/settings resolution, prompt guarantees, the full HTTP layer (in-process ASGI), SSE stream contract, and a determinism assertion — the same command must classify identically three times in a row.
- **20-flow live E2E harness** against the real backend + Ollama: every action's approve *and* deny paths, memory save→recall→delete, upload + summarize, voice pipeline, activity log, delete-all.

Running Luna

From the packaged build (Windows 10/11, one-time setup ≈ 5 min):

1. Install Ollama → <https://ollama.com>
2. `ollama pull llama3.2:3b` (one-time, ~2 GB)
`ollama pull nomic-embed-text` (one-time, ~274 MB)
3. Make sure Ollama is running: `ollama serve`
4. Unzip the Luna folder anywhere and run `Luna.exe`

From source: `uv run python -m luna.main` · **Build:** `powershell -File build.ps1` → `dist\Luna\Luna.exe`
(onedir — faster startup, fewer antivirus false positives).